

Informe t?cnico - Detector de grietas

MVP web de evaluaci?n visual preliminar de paredes

Repositorio: serpafabio121-pixel/proyecto-i1 | Documento generado el 12/09/2026

1. Resumen ejecutivo

El proyecto es una aplicaci?n web construida con Python y Streamlit para cargar fotos o videos de paredes y obtener una evaluaci?n visual preliminar. Devuelve clasificaci?n, confianza heur?stica, hallazgos, recomendaci?n y nivel orientativo de precauci?n ante sismo.

La aplicaci?n no sustituye una inspecci?n presencial, no certifica la seguridad de una estructura y no predice terremotos. Sus resultados sirven para ordenar evidencias y decidir cu?ndo solicitar una revisi?n profesional.

2. Respuesta expl?cita sobre entrenamiento e im?genes

El MVP actual NO fue entrenado con im?genes y NO utiliza un modelo de aprendizaje autom?tico para producir la clasificaci?n. No hay un conjunto de entrenamiento conectado al flujo actual, no se calculan pesos de una red neuronal y no se consulta una API de inteligencia artificial.

El resultado se obtiene con una heur?stica determinista de visi?n artificial programada en app/home.py. La misma imagen produce esencialmente el mismo resultado bajo las mismas condiciones; cambiar iluminaci?n, enfoque, encuadre o textura puede cambiar los indicadores.

El repositorio original conten?a modelos hist?ricos H5, pero fueron excluidos del flujo del MVP actual porque no son necesarios para la aplicaci?n heur?stica, dificultaban el despliegue y pod?an arrastrar dependencias pesadas. Por tanto, no debe afirmarse que este MVP tenga precisi?n validada como un modelo entrenado.

3. Lenguajes, formatos y estructura

Python es el lenguaje principal: interfaz, servidores, procesamiento multimedia, clasificaci?n y almacenamiento.

HTML y CSS aparecen a trav?s de Streamlit y bloques CSS embebidos para el tema oscuro y el dise?o responsive. JavaScript es generado por Streamlit/WebRTC; no existe un frontend JavaScript independiente.

TOML configura Streamlit en .streamlit/config.toml; YAML configura el workflow de GitHub Actions; Markdown documenta el proyecto en README.md.

Entrada principal: app/main.py. Interfaz y l?gica: app/home.py. Lanzadores: start_app.py, start_mobile.py y start_public.py. Dependencias: requirements.txt. Runtime de despliegue: runtime.txt.

4. Dependencias y procedencia

streamlit: interfaz web y servidor Python; PyPI: <https://pypi.org/project/streamlit/>.

numpy: arreglos y c?lculos num?ricos; PyPI: <https://pypi.org/project/numpy/>.

pillow: lectura y normalizaci?n de im?genes; PyPI: <https://pypi.org/project/Pillow/>.

opencv-python-headless: escala de grises, filtros, morfolog?a, bordes, videos y componentes conectados; PyPI: <https://pypi.org/project/opencv-python-headless/>.

streamlit-webrtc: c?mara y grabaci?n desde navegador; PyPI: <https://pypi.org/project/streamlit-webrtc/>.

av: PyAV para contenedores y fotogramas multimedia; PyPI: <https://pypi.org/project/av/>.

cryptography: certificado HTTPS local opcional; PyPI: <https://pypi.org/project/cryptography/>.

cloudflared es opcional y no se instala con pip; crea un túnel público temporal para usar la aplicación desde otra red.

5. Flujo de una foto

1) El usuario carga o captura una imagen. 2) Pillow la abre, corrige orientación EXIF y la convierte a RGB. 3) El usuario pulsa Analizar foto. 4) La imagen se reduce hasta 640 px para controlar coste. 5) OpenCV calcula señales locales. 6) Se obtiene crack_score. 7) Se muestra clasificación, riesgo, hallazgos y recomendación. 8) Opcionalmente se guarda una copia en data/uploads y se ofrece descarga.

6. Lógica de detección visual

La imagen se convierte a escala de grises. Se estima el fondo local mediante desenfoque gaussiano. La diferencia entre fondo e imagen resalta zonas oscuras.

Se aplica black-hat morfológico con un elemento elíptico de 11x11 para resaltar líneas oscuras. Se aplica cierre morfológico 3x3 para unir segmentos cercanos. Canny con umbrales 45 y 130 calcula bordes.

Se calculan tres señales: dark_ratio (porcentaje de patrón oscuro), edge_ratio (porcentaje de bordes) y elongated_ratio (área de componentes largos/estrechos). Un componente cuenta como alargado si tiene área ≥ 8 , longitud ≥ 16 y relación longitud/anchura ≥ 2 .

El índice usado por el MVP es: $\text{crack_score} = \min(1.0, \text{dark_ratio} * 1.25 + \text{edge_ratio} * 0.55 + \text{elongated_ratio} * 0.70)$. Es una puntuación visual interna, no una probabilidad de daño.

7. Clasificación y lógica de riesgo

La clasificación se interpreta así: score < 0.02 = Sin grieta evidente; score < 0.28 = Leve; score < 0.48 = Advertencia; score < 0.70 = Media; score ≥ 0.70 = Severa / grave.

La aplicación deriva un nivel preventivo ante sismo: Sin grieta evidente -> Bajo; Leve o Advertencia -> Moderado; Media -> Alto; Severa / grave -> Crítico.

Bajo significa que no se observaron señales claras en esa foto, no que el edificio esté certificado. Moderado indica seguimiento. Alto recomienda inspección profesional. Crítico recomienda alejarse de la zona y solicitar atención prioritaria.

El riesgo no predice un sismo ni calcula resistencia estructural. Grietas diagonales, ramificadas, que crecen, acompañadas de deformación, desprendimiento, filtración o ruidos requieren criterio profesional aunque el algoritmo marque bajo.

8. Videos

OpenCV lee el video localmente y muestrea hasta ocho fotogramas representativos distribuidos entre el inicio y el final. Cada fotograma pasa por la misma heurística de foto.

Los resultados se combinan priorizando el percentil 75 y el peor score observado. El resultado es orientativo: una grieta que aparece entre muestras puede no detectarse. Se admiten MP4, MOV, AVI, WebM y M4V, con límite aproximado de 50 MB.

9. PC, celular y URLs

start_app.py inicia una única URL local: el PC puede usar http://localhost:8501 y el celular la IP del PC en el mismo puerto. El enlace móvil puede llevar ?device=mobile para mostrar solo carga de foto/video.

En modo móvil se ocultan cámara y grabación directa según la solicitud de producto; el usuario carga imágenes o videos. El PC conserva carga, cámara, grabación, selector frontal/trasero y análisis.

Para usar distintas redes se necesita `start_public.py` con cloudflared o un despliegue HTTPS permanente (por ejemplo Streamlit Community Cloud). GitHub almacena código, pero no ejecuta la app.

10. Instalación y ejecución

Windows: `python -m pip install -r requirements.txt` y luego `python start_app.py`. Para Streamlit Cloud se usa Python 3.11, `app/main.py` y `requirements.txt`. La terminal debe permanecer abierta en ejecución local.

El lanzador verifica dependencias y muestra la IP actual. Si falta Streamlit, debe instalarse con el mismo Python que ejecuta `start_app.py`.

11. Almacenamiento y privacidad

El procesamiento de fotos y videos se realiza localmente en el proceso Streamlit. No se envían imágenes a una API de IA. Las copias explícitamente guardadas se escriben en `data/uploads` con nombres únicos. Los videos de grabación usan un archivo temporal WebM y se limpian al terminar cuando el sistema lo permite.

En servicios efímeros como Streamlit Cloud, `data/uploads` puede desaparecer al reiniciar. Descarga los archivos que deban conservarse. No se guardan credenciales ni claves en el repositorio.

12. Limitaciones y validación

Puede confundir sombras, suciedad, juntas, humedad, cables, reflejos, textura o mala iluminación con grietas. También puede no detectar una grieta si la imagen está borrosa, demasiado lejos o mal encuadrada.

Las validaciones realizadas incluyen compilación Python, health check de Streamlit, smoke tests con imagen limpia, imagen con línea oscura, imagen con múltiples líneas y video sintético, además de resolución de `requirements.txt`.

La validación sintética no equivale a una validación científica de exactitud. Para un sistema de ingeniería se necesitaría un dataset etiquetado, separación entrenamiento/validación/prueba, métricas (precision, recall, F1, sensibilidad, especificidad), evaluación por tipo de superficie y revisión de profesionales.

13. Conclusión

El resultado entregado es un MVP funcional, local, explicable y desplegable. Su valor está en ofrecer una primera señal visual y recomendaciones de seguridad, no en reemplazar una inspección estructural. La evolución natural sería recopilar un dataset autorizado y etiquetado, entrenar un modelo específico y validarlo con protocolos de ingeniería antes de usarlo para decisiones críticas.